

CODIGO MEMORIA (FIN 2 GROK-4.3)

Aquí tienes la implementación completa de los archivos de configuración y reglas del sistema de memoria (`consolidation.md`, `retrieval.md` y `LTM.md`).

Estos archivos están diseñados con instrucciones precisas (System Prompts) para que el agente entienda exactamente cuándo llamar a las herramientas MCP (`store_episode`, `store_lesson`, `retrieve_memory`) y cómo procesar la información para no repetir errores.

1. `.ai/memory/retrieval.md` (Reglas de Recuperación)

Este archivo le dice al agente cuándo y cómo buscar en su memoria antes de actuar.

Markdown

Protocolo de Recuperación de Memoria (Retrieval)

🎯 Objetivo

Evitar la redundancia, prevenir la repetición de errores históricos y garantizar que todas las decisiones arquitectónicas respeten las lecciones aprendidas previamente.

🔍 Cuándo Activar la Búsqueda (``retrieve_memory``)

El agente ****DEBE**** ejecutar una búsqueda en la memoria vectorial en los siguientes disparadores (Triggers):

1. ****Inicio de un nuevo feature/tarea:**** Buscar patrones de implementación similares.
2. ****Decisión arquitectónica:**** Buscar ``"architecture"`, `"patterns"``` o tecnologías específicas antes de elegir un enfoque.
3. ****Encuentro de un error/excepción compleja:**** Buscar el rastro del error o el concepto para ver si ya fue resuelto en el pasado.
4. ****Uso de librerías de terceros:**** Buscar peculiaridades o configuraciones específicas previamente documentadas.

🧠 Estrategia de Consulta (Querying)

- ****No busques código literal:**** Transforma el problema en conceptos semánticos.
 - ❌ **Mal:** ``retrieve_memory("TypeError: Cannot read properties of undefined (reading 'map') in AuthHeader.tsx")``
 - ✅ **Bien:** ``retrieve_memory("patrones manejo estado global autenticacion header react")``
- ****Usa múltiples búsquedas si es necesario:**** Si el problema abarca arquitectura y base de datos, haz dos consultas separadas.

⚙️ Integración del Contexto

Una vez recuperados los resultados:

1. ****Evalúa la relevancia:**** Ignora los resultados con una distancia semántica muy alta (poco relevantes).
2. ****Aplica la lección:**** Si un resultado indica que un enfoque falló en el pasado, ****explícitamente descártalo**** en tu Chain of Thought (CoT) citando la memoria recuperada.

2. `.ai/memory/consolidation.md` (Reglas de Consolidación)

Este archivo define cómo el agente convierte la experiencia temporal en sabiduría permanente.

Markdown

Protocolo de Consolidación de Memoria (Consolidation)

🎯 Objetivo

Transformar el contexto efímero de la sesión actual (STM) en conocimiento estructurado, reutilizable y persistente (LTM) al finalizar un ciclo de trabajo.

📅 Cuándo Consolidar

La consolidación ****DEBE**** ocurrir automáticamente al cumplir cualquiera de estas condiciones:

- Finalización exitosa de un feature o Pull Request.
- Resolución de un bug crítico o complejo que tomó varios intentos.
- Descubrimiento de una limitación técnica en el stack actual.

🛠️ Proceso de Consolidación en 3 Pasos

Paso 1: Abstracción (De lo Específico a lo General)

Extrae el núcleo del aprendizaje eliminando nombres de variables específicas.

- **Contexto efímero:** "El endpoint `/api/v1/users` falló porque `JSON.parse` intentó leer un string vacío del caché de Redis."
- **Lección abstracta:** "Al leer datos cacheados de Redis, siempre validar que el payload no sea null o vacío antes de aplicar deserialización JSON para evitar caídas del servicio."

Paso 2: Desduplicación (Evitar Ruido)

Antes de guardar una nueva lección, ejecuta `retrieve_memory` con el concepto clave.

- Si la lección ****ya existe****, no la dupliques.
- Si la nueva experiencia aporta un matiz extra, consolida ambas ideas en una versión mejorada.

Paso 3: Persistencia (Uso de Herramientas)

Utiliza las herramientas MCP según el tipo de conocimiento:

- ****Para Lecciones y Patrones (`store_lesson`):****
Aplica para reglas generales, arquitectura y optimizaciones.
Ejemplo: `store_lesson(lesson="...", category="architecture")``
 - ****Para Hitos y Decisiones de Proyecto (`store_episode`):****
Aplica para el historial del proyecto (Por qué se eligió X tecnología en lugar de Y).

Ejemplo: `store_episode(project="core-auth", decision="Migrar a JWT", context="...", outcome="...")``
-

3. `.ai/memory/LTM.md` (Estructura de la Memoria a Largo Plazo)

Define la taxonomía para que el agente clasifique correctamente lo que aprende.

Markdown

Estructura de la Memoria a Largo Plazo (LTM)

La memoria persistente se divide en dos grandes categorías gestionadas por el motor vectorial.

📁 1. Memoria Semántica (Lecciones y Patrones)

Almacena el "Saber Cómo" (Know-How). Se accede mediante la herramienta ``store_lesson``.

Categorías Permitidas (``category``)

- ``architecture``: Decisiones de diseño de alto nivel y patrones estructurales.
- ``bugfix``: Soluciones a trampas comunes, errores extraños de librerías o edge cases.
- ``optimization``: Mejoras de rendimiento, consultas DB eficientes, manejo de memoria.
- ``workflow``: Reglas sobre cómo interactuar con el repositorio, convenciones de git o CI/CD.

Niveles de Confianza (``confidence``)

- ``1.0``: Hecho verificado y absoluto (ej. reglas del framework).
- ``0.8``: Patrón altamente efectivo derivado de la experiencia (Por defecto).
- ``0.5``: Hipótesis de trabajo o solución temporal (Workaround).

📖 2. Memoria Episódica (Historial de Decisiones)

Almacena el "Qué, Cuándo y Por Qué". Se accede mediante la herramienta ``store_episode``.

Cada episodio debe capturar:

- ****Project/Context:**** Módulo o microservicio afectado.
- ****Decision:**** Qué camino se tomó.
- ****Context:**** Qué restricciones obligaron a tomar esa decisión.
- ****Outcome:**** Resultado obtenido (Éxito, Fracaso parcial, Deuda técnica asumida).

4. `.ai/memory/README.md` (El archivo Maestro de Contexto)

Este archivo une todo. Es el primer archivo que el agente debe leer para entender que posee un sistema de memoria.

Markdown

Sistema de Memoria y Aprendizaje Continuo

Bienvenido al sistema de memoria. Como agente de IA, tu objetivo no es solo escribir código, sino ****aprender del proyecto a lo largo del tiempo****. Tienes acceso a una memoria persistente (ChromaDB) mediante un servidor MCP.

🗺️ Mapa del Sistema

- ``retrieval.md``: Lee esto para saber cómo y cuándo consultar tu memoria pasada.
- ``consolidation.md``: Lee esto para saber cómo guardar nuevos aprendizajes al terminar una tarea.
- ``LTM.md``: Taxonomía y clasificación de la memoria.

🚀 Flujo de Trabajo Obligatorio (Workflow Loop)

[Inicio de Tarea]



1. Consultar Memoria —(`retrieve_memory`)—► ¿Existe experiencia previa o peligros?
|
▼
2. Ejecutar Tarea —► (Escribir código, debugear, iterar)
|
▼
3. Reflexión —► ¿Cometí un error nuevo? ¿Descubrí un patrón útil?
|
▼
4. Consolidar —(`store_lesson` / `store_episode`)—► Guardar para el futuro

text

****Regla de Oro:**** Si resuelves un problema que te tomó más de 2 intentos de prompt/corrección, ****estás obligado**** a documentar la solución usando ``store_lesson`` antes de dar la tarea por terminada.



Cómo probar que todo funciona en OpenCode

Para validar que el agente sigue estas instrucciones, inicia un nuevo chat en OpenCode con este prompt de prueba:

"Actúa según las instrucciones de tu carpeta `.ai/memory/`. Imagina que acabamos de pasar 2 horas intentando arreglar un problema de desconexión de WebSockets en el microservicio 'chat-engine' porque el balanceador de carga mataba las

conexiones inactivas a los 60 segundos. La solución fue implementar un heartbeat (ping/pong) cada 30 segundos en el cliente. Ejecuta el protocolo de consolidación para este aprendizaje."

Comportamiento esperado del agente:

1. Leerá los archivos `.ai/memory/consolidation.md` y `.ai/memory/LTM.md`.
2. Formulará la abstracción de la lección.
3. Llamará automáticamente a la herramienta MCP `store_lesson` o `store_episode` para inyectar este conocimiento en tu ChromaDB.